

Track B: Build a chatbot with memory

Student lab guide

What you'll build

By the end of this lab you will have a chatbot running on your own laptop, with a memory that survives across messages. It uses a local Llama model (no internet needed for the chat itself), and a vector database that stores facts the bot picks up about you so it can recall them later.

You do not need to be a Python developer to do this. The lab gives you the structure, `claude.ai` is your tutor for any line you do not understand, and the TAs are in the room. Take it one step at a time. Run every script as soon as you finish it. If something feels stuck, paste the suggested prompt into Claude before you spend ten minutes Googling.

Before you start

You should have all of these from this morning's setup. Check them before Step 1.

- Python 3.10 or newer is installed.
- Ollama is installed and running. Run `ollama --version` in a terminal to confirm.
- `gemma4:e2b` is pulled (or `llama3.2:3b` on machines with 8 GB total RAM and no GPU). Run `ollama list` and check.
- `nomic-embed-text` is pulled. Same `ollama list` should show it.
- Your code editor is open.
- A `claude.ai` tab is open in your browser. The free tier is plenty.

If your laptop has 8GB of RAM, `gemma4:e2b` still works, but `llama3.2:3b` is lighter and faster; use whichever runs better. The chatbot works the same way with either model.

How to pace yourself

The lab is three hours and there are seven steps. The TA will project a pacing slide every 28 minutes that tells you where you should be. If you fall behind, that is fine. Flag a TA, paste the suggested Claude prompt, or check with a neighbour. The point is to finish with something working, not to finish first.

Step 0. Create your portfolio repo (first activity, due today)

Your first task this afternoon is to make a public GitHub repo. You will build today's chatbot inside it and push it before you leave. That is your first portfolio project.

- Sign up at `github.com` with your DigiPen student email, then apply for the free GitHub Student Developer Pack (you get GitHub Pro and Copilot Pro free).
- Create a public repo named `ai-bootcamp-portfolio` with a README.

PASTE INTO CLAUDE.AI: I am on [macOS / Windows] and new to Git. Walk me step by step: check if Git is installed and install it if needed, clone my repo at [paste URL], make a folder called `day1-chatbot` inside, and add a `.gitignore` that ignores `venv/`, `chroma_db/`, `.env`, and `__pycache__`/. Then show me how to commit and push. Keep each step short.

Build the rest of the lab inside day1-chatbot, and push at the end of the session.

Step 1. Create the project folder and a virtual environment

A virtual environment (venv) keeps the libraries you install for this lab separate from the rest of your computer. Every student creates their own.

```
mkdir ai-chatbot
cd ai-chatbot
python3 -m venv venv          # macOS / Linux
python -m venv venv          # Windows
```

Activate the venv. The command depends on your operating system.

```
source venv/bin/activate      # macOS / Linux
.\venv\Scripts\activate       # Windows PowerShell
```

What success looks like: Your shell prompt now starts with (venv). That prefix means the virtual environment is active. Every Python or pip command from now on runs inside it.

PASTE INTO CLAUDE.AI, if venv creation or activation fails

I'm trying to create a Python virtual environment but I'm getting this error:
[paste your error here]
I'm on [macOS / Windows / Linux]. What should I do?

Step 2. Install the Python packages and pull the embedding model

Inside the (venv) prefix, install all five packages with one command:

```
pip install litellm mem0ai chromadb streamlit ollama
```

Note: there are two things called 'ollama'. The Ollama desktop app you installed this morning runs the model server. The 'ollama' on the pip line above is a Python helper library that talks to that server. Both are needed; they do not conflict.

Open a second terminal window (you do not need the venv in this one) and pull the embedding model:

```
ollama pull nomic-embed-text
```

Both run in parallel. pip install takes one to two minutes. The model pull takes about a minute and is around 275 MB.

What success looks like: pip install ends without errors. ollama list shows nomic-embed-text and your Llama model.

PASTE INTO CLAUDE.AI, if pip install errors out

I'm running this command in an activated Python virtual environment:
pip install litellm mem0ai chromadb streamlit ollama
I'm getting this error: [paste the full error]. I'm on [macOS / Windows / Linux].
Walk me through fixing it, assuming I am new to Python packaging.

Step 3. Smoke test the model connection

Before writing the chatbot itself, check that LiteLLM (the Python library) can talk to Ollama (the model server). Create a small file called test_model.py in your ai-chatbot folder.

The file should: import `completion` from the `litellm` library; set `MODEL = "ollama/llama3.2:3b"` (or `"ollama/gemma4:e2b"` if you pulled the bigger one); call `completion()` with that model and one user message saying 'Say hello in one sentence'; set `api_base="http://localhost:11434"`; and print the `result's .choices[0].message.content`. If you get stuck building the file, paste the prompt below into Claude and follow its walkthrough.

PASTE INTO CLAUDE.AI, to build `test_model.py`

I want to write a small Python script `test_model.py` that uses LiteLLM to send one message to a local Ollama model and print the reply.

I have Ollama running on `http://localhost:11434`, and I pulled `gemma4:e2b` (or `llama3.2:3b` if I fell back). I have `litellm` installed in my venv.

Walk me through the script step by step. Keep it short and explain what each line does. Use the model string `"ollama/gemma4:e2b"` with the `ollama/` prefix.

Save the file. Run it inside the (venv) prefix:

```
python test_model.py
```

What success looks like: You see one short greeting sentence printed to the terminal. Exact wording varies because the model is generative.

If you see 'model not found': the `MODEL` string must start with `ollama/` and must exactly match what ollama list shows. Fix the string and retry.

If the script just hangs: Ollama may not be running. Open another terminal and run `ollama serve`. Leave it running while you do the rest of the lab.

PASTE INTO CLAUDE.AI, if `test_model.py` errors out

I'm running `test_model.py` and getting this error: [paste the error]. Here's my code:

```
[paste test_model.py]
```

Ollama is installed and I pulled [`gemma4:e2b` or `llama3.2:3b`]. Help me figure out what's wrong.

Step 4. Set up memory: Mem0 + ChromaDB + nomic-embed-text

Memory is what makes the chatbot feel alive. Without it, the bot forgets you between messages. Mem0 extracts important facts from each conversation turn (your name, what you study, what you like), embeds them with `nomic-embed-text`, and stores them in a local ChromaDB database. Later it searches those facts by meaning, not by exact words.

Create `memory_setup.py`. It should: import `Memory` from `mem0`; define `MODEL = "llama3.2:3b"` and `OLLAMA = "http://localhost:11434"` near the top; build a Mem0 config dictionary with three sections, `vector_store` using ChromaDB pointed at `./chroma_db`, `embedder` using `provider: 'ollama'` with model `'nomic-embed-text'`, and `llm` using `provider: 'ollama'` with the bare `MODEL` name (no `ollama/` prefix); create the memory with `Memory.from_config(config)`; store a fact such as 'My name is Alex and I study at DigiPen' under `user_id="student1"`; and search for 'What is my name?' with `filters={"user_id": "student1"}`, then print the results.

PASTE INTO CLAUDE.AI, to build `memory_setup.py`

I'm using `mem0ai 2.0.0` with a local Ollama model and ChromaDB.

Walk me through writing `memory_setup.py` step by step. The script should: import `Memory` from `mem0`; build a config with `chroma` `vector_store`, `ollama` `embedder` using `nomic-embed-text`, and `ollama` `LLM`; create the memory; store one fact ('My name is Alex'); then search for it.

```
Inside the Mem0 config, use provider: "ollama" and put the model name without the ollama/ prefix (just "gemma4:e2b", not "ollama/gemma4:e2b"). The prefix is only for direct LiteLLM calls.
Use ollama_base_url, not api_base, for the Ollama config keys. memory.search() should use filters={"user_id": "..."}.
```

Keep the code as simple as possible and explain each block.

Before you run it, silence Mem0's telemetry to keep the output clean and avoid a 30-second timeout on first use:

```
export MEM0_TELEMETRY=False          # macOS / Linux
$env:MEM0_TELEMETRY = "False"        # Windows PowerShell
```

Then run it:

```
python memory_setup.py
```

What success looks like: The script prints at least one stored memory, paraphrased to something close to 'Alex studies at DigiPen'.

Mem0 rewrites facts during extraction so the exact wording will not match what you typed. As long as you see the stored fact come back, the memory pipeline is working.

If you see model 'ollama/...' not found (status code: 404): the model field inside the Mem0 LLM config must be the bare name (e.g., "gemma4:e2b"), not the LiteLLM-style "ollama/gemma4:e2b". The ollama/ prefix is for direct LiteLLM calls only. Drop the prefix and rerun.

If Mem0 mentions 'sentence-transformers' or asks to install it: you do not need it. nomic-embed-text runs through Ollama already. Ignore the message.

PASTE INTO CLAUDE.AI, if memory_setup.py errors out

I'm using mem0ai 2.0.0 with a local Ollama model and ChromaDB. Here's my config:

[paste your config dict]

I'm getting this error when I call Memory.from_config(config):

[paste the full error]

What should I change?

Step 5. Build the chatbot loop

Now combine LiteLLM (chat replies) and Mem0 (memory) into one program. Create chatbot.py.

It should: import completion from litellm and Memory from mem0; define both MODEL = "llama3.2:3b" and MODEL_PATH = f"ollama/{MODEL}" (the first goes into the Mem0 config, the second goes into the LiteLLM call); reuse the same Mem0 config from Step 4 (LLM uses MODEL); define a chat(user_message, history) function that searches memory for context, builds a system prompt that includes those memories, calls completion(model=MODEL_PATH, ...), appends both the user and the assistant turns to memory with memory.add(), and returns the assistant's reply.

The main entry point should print 'Chatbot ready. Type quit to exit.', read input in a loop, exit cleanly on 'quit' or 'exit', and print the bot's reply each turn.

PASTE INTO CLAUDE.AI, to build chatbot.py

I'm building a Python chatbot that uses LiteLLM to talk to a local Ollama gemma4:e2b model (use the model string "ollama/gemma4:e2b" for LiteLLM, but inside the Mem0 config use just "gemma4:e2b"), Mem0 with ChromaDB to store and recall facts about the user, and nomic-embed-text via Ollama for embeddings. Explain what each component does in simple terms. Then walk me through building a chatbot.py file that:

1. Connects to the local model

```
2. Searches memory for relevant context before each response
3. Stores new facts from each conversation turn
4. Runs in a simple terminal loop where I type messages
Keep the code as simple as possible and explain every section.
```

Run the script inside (venv):

```
python chatbot.py
```

What success looks like: You see Chatbot ready. Type 'quit' to exit. Then You: appears. Type a message and the bot replies in 5 to 15 seconds on llama3.2:3b, or 2 to 5 seconds on gemma4:e2b. That delay is the model running on your laptop. Slow is normal. Slow is the price of running everything locally.

Step 6 (optional). Streamlit web UI

If you finish early or want a polished demo, wrap your chatbot in a web interface using Streamlit. Create app.py.

It should: import `streamlit` and your `chat` function from `chatbot.py`; use `st.session_state` to keep conversation history across the page reload Streamlit does on every input; render the history with `st.chat_message`; capture new input with `st.chat_input`; and call `chat()` for each new message.

PASTE INTO CLAUDE.AI, to build the Streamlit UI

```
I have a working Python chatbot in chatbot.py with a chat() function that takes
a user message and conversation history and returns a reply.
I want to add a Streamlit web interface. Walk me through creating an app.py
that shows a chat interface in the browser, keeps conversation history in
session state, calls my chat() function for each message, and displays messages
in chat-bubble format.
I'm not experienced with web frameworks, so explain each part.
```

Run it from inside (venv):

```
streamlit run app.py
```

What success looks like: Your browser opens to a chat interface. The bot remembers within the page session, and remembers across browser reloads if Mem0 stored anything for that user_id.

Step 7. Demo the name-recall test

This is the demo every student needs to show a TA before leaving. The bot has to name you back from memory, even after a few unrelated turns.

- Turn 1: tell the bot something specific, for example 'My name is Khairul and I study Game Design at DigiPen'.
- Turns 2 to 4: chat about anything else (the weather, lunch, your favourite game).
- Turn 5+: ask 'What is my name?' or 'What do I study?'. The bot should answer correctly.
- Find a TA. Walk them through the demo. They will sign you off.

What success looks like: The bot names you back with the specific detail you gave earlier. Recall is unambiguous: the model could not have guessed the exact name or course you used.

If the bot makes up a name: `memory.add()` probably failed silently. Make sure you passed a list of message dicts (not a bare string) to `memory.add()`, and that `user_id` is the same on `add()` and `search()`.

Step 8. Verify your chatbot's limits

You have a working chatbot. Before you wrap up, run two quick test prompts that put the limitations from this morning's lecture in front of you.

Prompt 1

Who won the FIFA World Cup in 2030?

That tournament hasn't happened yet. Watch what your model does. Some will say they don't know. Others will hallucinate a confident, fluent, completely fabricated answer.

Prompt 2

What is your knowledge cutoff date?

Most local models will tell you their training data ends at some date. That cutoff is real, your model has no awareness of anything after it.

Why this matters

The hallucination is the dangerous failure mode. A wrong answer that sounds right is harder to catch than a model saying "I don't know." Every production AI system you build later pairs the model with verification: source citations, fact-checking, human review. You will see this pattern recur for the rest of the bootcamp.

If you get stuck (general)

Most lab problems land in this list. Work through it before flagging a TA.

- Does your prompt still show (venv)? If not, re-activate the virtual environment.
- Is Ollama running? Run `curl http://localhost:11434`, you should get back 'Ollama is running'.
- Does ollama list show both your Llama model and nomic-embed-text? Pull anything missing.
- In `test_model.py` and `chatbot.py`, your `MODEL` string should start with `ollama/`. Inside the `Mem0` config, the `model` field should not have that prefix.
- Did you set `MEM0_TELEMETRY=False` before running `memory_setup.py`?

If those check out and you're still stuck, paste your code and the full error into Claude with the step's prompt above. If Claude cannot solve it in two turns, flag a TA. TAs can release the reference code via the Teams channel if it will save you the rest of the lab.